

1. Introduction

- Create a new Rails app using rails new blog_app generator command. (5 mins)
- Identify what Rails generates by default (5 mins)
- Push initial setup to a Git repository and raise a PR. (5 mins)
- Explore Rails Guides: Getting Started guide (15 mins)

```
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails$ git checkout -b feature/blog_app
Switched to a new branch 'feature/blog_app'
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails$ rails new blog_app
  create
  create  README.md
  create  Rakefile
  create  .ruby-version
  create  config.ru
  create  .gitignore
  create  .gitattributes
  create  Gemfile
  run    git init -b main from "."
Initialized empty Git repository in /home/hp/Desktop/Ruby_on_Rails/blog_app/.git/
  create  app
  create  app/assets/stylesheets/application.css
  create  app/controllers/application_controller.rb
  create  app/helpers/application_helper.rb
  create  app/jobs/application_job.rb
  create  app/mailers/application_mailer.rb
  create  app/models/application_record.rb
  create  app/views/layouts/application.html.erb
  create  app/views/layouts/mailer.html.erb
  create  app/views/layouts/mailer.text.erb
  create  app/views/pwa/manifest.json.erb
  create  app/views/pwa/service-worker.js
  create  app/assets/images
  create  app/assets/images/.keep
  create  app/controllers/concerns/.keep
  create  app/models/concerns/.keep
  create  bin
  create  bin/brakeman
  create  bin/bundler-audit
  create  bin/ci
  create  bin/dev
  create  bin/rails
  create  bin/rake
  create  bin/rubocop
  create  bin/setup
  create  bin/thrust
  create  Dockerfile
  create  .dockerignore
  create  bin/docker-entrypoint
  create  .rubocop.yml
  create  .github/workflows
  create  .github/workflows/ci.yml
  create  .github/dependabot.yml
```

```

create log
create log/.keep
create public
create public/400.html
create public/404.html
create public/406-unsupported-browser.html
create public/422.html
create public/500.html
create public/icon.png
create public/icon.svg
create public/robots.txt
create script
create script/.keep
create tmp
create tmp/.keep
create tmp/pids
create tmp/pids/.keep
create vendor
create vendor/.keep
create test/fixtures/files
create test/fixtures/files/.keep
create test/controllers
create test/controllers/.keep
create test/mailers
create test/mailers/.keep
create test/models
create test/models/.keep
create test/helpers
create test/helpers/.keep
create test/integration
create test/integration/.keep
create test/test_helper.rb
create storage
create storage/.keep
create tmp/storage
create tmp/storage/.keep
remove config/initializers/cors.rb
remove config/initializers/new_framework_defaults_8_1.rb
run bundle install --quiet
run bundle lock --add-platform=x86_64-linux
Writing lockfile to /home/hp/Desktop/Ruby_on_Rails/blog_app/Gemfile.lock
Fetching gem metadata from https://rubygems.org/.....
Resolving dependencies...
rails importmap:install
apply /home/hp/.rbenv/versions/3.2.2/lib/ruby/gems/3.2.0/gems/importmap-rails-2.2.3/lib/install/installer.rb:10: warning: Using the last argument for keyword arguments in `new' is deprecated; the `:tag' option may be preferred.
Add Importmap include tags in application layout

```



Add a title

Added the structure of blog_app

Add a description

Write

Preview

H B I

Add your description here...

Markdown is supported Paste, drop, or click to add files

Create pull request

Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

2. Folder Structure & Request Lifecycle

- Use the rails scaffold command to create a blog app, with a blog model(title: string, body: text) (10 mins)
- Explore Rails Structure: identify the purpose of app/controllers, app/models, app/views, and config/routes.rb, db/migrate (10 mins)
- Start the server and trace the request flow for GET /blogs, check logs. Identify route, controller action, model interaction, response type (10 mins)

```

[new branch] feature/blog_app → feature/blog_app
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails$ cd blog_app/
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails generate scaffold Blog title:string body:text
  invoke  active_record
  create  db/migrate/20260117070828_create_blogs.rb
  create  app/models/blog.rb
  invoke  test_unit
  create  test/models/blog_test.rb
  create  test/fixtures/blogs.yml
  invoke  resource_route
  route   resources :blogs
  invoke  scaffold_controller
  create  app/controllers/blogs_controller.rb
  invoke  erb
  create  app/views/blogs
  create  app/views/blogs/index.html.erb
  create  app/views/blogs/edit.html.erb
  create  app/views/blogs/show.html.erb
  create  app/views/blogs/new.html.erb
  create  app/views/blogs/_form.html.erb
  create  app/views/blogs/_blog.html.erb
  invoke  resource_route
  invoke  test_unit
  create  test/controllers/blogs_controller_test.rb
  invoke  helper
  create  app/helpers/blogs_helper.rb
  invoke  test_unit
  invoke  jbuilder
  create  app/views/blogs/index.json.jbuilder
  create  app/views/blogs/show.json.jbuilder
  create  app/views/blogs/_blog.json.jbuilder
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$

```

The rails generate scaffold Blog title:string body:text command is used to quickly create a complete blog feature in a Rails application. Instead of manually writing code for the database, model, controller, routes, and user interface, this single command generates all of them together following Rails conventions. It sets up a Blog model with title and body fields, creates the database migration to store blogs, adds a controller to handle requests, builds pages to create, view, edit, and delete blog posts, and wires everything through routes. In simple terms, it gives you a ready-to-use blog CRUD feature so you can focus on understanding or improving the logic rather than building everything from scratch.

Exploring Rails Folder Structure

In a Ruby on Rails application, the code is organized using the MVC (Model–View–Controller) pattern to keep responsibilities clearly separated. The app/controllers folder contains controller files that handle incoming HTTP requests. A controller decides what should happen when a user visits a URL, such as fetching data from the database or saving new data, and then chooses the appropriate view to display.

The app/models folder contains model classes that represent the application's data and business rules. Models communicate with the database using ActiveRecord and are responsible for creating, reading, updating, and deleting records. For example, a Blog model maps directly

to the `blogs` table in the database and allows the application to interact with blog records using Ruby code instead of SQL.

The `app/views` folder contains templates that generate the HTML sent to the browser. Views focus only on presentation and display the data provided by the controller. Each controller usually has its own folder inside `views`, and the files inside it correspond to controller actions such as `index`, `show`, or `edit`.

The `config/routes.rb` file defines how URLs are mapped to controller actions. When a request comes in, Rails first checks this file to determine which controller and action should handle that request. It acts as the entry point for all HTTP requests.

The `db/migrate` folder stores migration files, which describe changes to the database structure over time. Each migration records a specific change, such as creating a table or adding a column, and allows the database schema to stay consistent across different environments.

Tracing the Request Flow for GET `/blogs`

After starting the Rails server using `rails server`, when a user visits the URL `/blogs` in the browser, a GET request is sent to the Rails application. Rails first looks into `config/routes.rb` to find a matching route. The route `resources :blogs` maps the GET `/blogs` request to the `index` action of `BlogsController`.

`resources :blogs` is a **Rails routing shortcut** that automatically creates all the standard URL routes needed to perform CRUD operations for a Blog resource.

Once the route is matched, Rails calls the `index` method inside `BlogsController`. Inside this action, the controller typically interacts with the Blog model by calling something like `Blog.all` to fetch all blog records from the database. This is where the model communicates with the database and returns the data as Ruby objects.

After the data is fetched, the controller automatically renders the `index.html.erb` view located in `app/views/blogs`. This view uses the data provided by the controller to generate an HTML page. Finally, Rails sends this HTML as the response back to the browser.

When checking the server logs in the terminal, you can see this entire flow clearly: the incoming GET request, the route being matched, the controller action being executed, the database query

performed by the model, and the HTML response being rendered. The response type for GET /blogs is HTML, which is displayed in the browser.

3. Models

- Create a Comment model (10 mins)
- Explore what helper methods ActiveRecord adds automatically to both the model and the record
- Add a published: boolean column using migration to the blog model (5 mins)
- Explore migration helpers like add_index, rename_column
- Add a published scope to blog model (5 mins)
- Add validations for blog, comment models. Comments can only be added for published blogs. (20 mins)
- Verify validations using a valid method on the console. (20 mins)
- Explore other validation helpers
- Explore other lifecycle callbacks
- Use AR method on rails console: where, find_by, limit
- Explore order, joins, includes, eager_load, Identify the difference between includes and eager_load
- Explore seeds, and try to create 20 blogs(10 published, 10 unpublished) with comments.

```
Exiting
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails generate model Comment body:text blog:references
  invoke  active_record
  create  db/migrate/20260117074626_create_comments.rb
  create  app/models/comment.rb
  invoke  test_unit
  create  test/models/comment_test.rb
  create  test/fixtures/comments.yml
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails db:migrate
== 20260117070828 CreateBlogs: migrating =====
-- create_table(:blogs)
   -> 0.0032s
== 20260117070828 CreateBlogs: migrated (0.0033s) =====

== 20260117074626 CreateComments: migrating =====
-- create_table(:comments)
   -> 0.0055s
== 20260117074626 CreateComments: migrated (0.0056s) =====
```

```
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails generate migration AddPublishedToBlogs published:boolean
  invoke  active_record
  create  db/migrate/20260117075250_add_published_to_blogs.rb
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails db:migrate
== 20260117075250 AddPublishedToBlogs: migrating =====
-- add_column(:blogs, :published, :boolean)
   -> 0.0028s
== 20260117075250 AddPublishedToBlogs: migrated (0.0029s) =====

hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$
```

What are migration helpers?

Migration helpers are **built-in Rails methods** that help you change the database structure using **Ruby**, instead of writing raw SQL. They exist to make database changes **safe, readable, and reversible**.

Instead of worrying about database-specific SQL syntax, you describe *what you want*, and Rails handles *how to do it*.

```
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails generate migration RenameBodyToContentInBlogs
    invoke  active_record
    create  db/migrate/20260117100558_rename_body_to_content_in_blogs.rb
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails db:migrate
== 20260117100558 RenameBodyToContentInBlogs: migrating =====
-- add_index(:blogs, :published)
   -> 0.0054s
-- rename_column(:blogs, :body, :content)
   -> 0.0340s
== 20260117100558 RenameBodyToContentInBlogs: migrated (0.0398s) =====

hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails console
Loading development environment (Rails 8.1.2)
blog-app(dev):001> Blo
(blog-app):1:in `<main>': uninitialized constant Blo (NameError)

Blo
^^^
Did you mean? Blog
blog-app(dev):002> Blog.column_names
=> ["id", "content", "created_at", "published", "title", "updated_at"]
blog-app(dev):003>
```

```
blog-app(dev):003> draft = Blog.create!(title: "Draft", content: "Text", published: false)
blog-app(dev):004> comment = draft.comments.build(body: "Nice")
blog-app(dev):005> comment.valid?
blog-app(dev):006>
blog-app(dev):007> comment.errors.full_messages
blog-app(dev):008>
TRANSACTION (0.3ms) BEGIN immediate TRANSACTION /*application='BlogApp'*/
Blog Create (3.0ms) INSERT INTO "blogs" ("content", "created_at", "published", "title", "updated_at") VALUES ('Text', '2026-01-17 10:10:45.761303', FALSE, 'Draft', '2026-01-17 10:10:45.761303') RETURNING "id" /*application='BlogApp'*/
TRANSACTION (10.0ms) COMMIT TRANSACTION /*application='BlogApp'*/
(blog-app):4:in `<main>': undefined method `comments' for #<Blog id: 1, content: "Text", created_at: "2026-01-17 10:10:45.761303000 +0000", published: false, title: "Draft", updated_at: "2026-01-17 10:10:45.761303000 +0000"> (NoMethodError)
Did you mean? committed!
blog-app(dev):009>
```

```
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ nano app/models/blog.rb
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails console
Loading development environment (Rails 8.1.2)
blog-app(dev):001> blog = Blog.new
=> #<Blog:0x00007d1951618b80 id: nil, content: nil, created_at: nil, published: nil, title: nil, updated_at: nil>
blog-app(dev):002> blog.valid?
blog-app(dev):003> blog.errors.full_messages
=> ["Title can't be blank", "Title is too short (minimum is 5 characters)", "Content can't be blank"]
blog-app(dev):004>
```

Explore other lifecycle callbacks

Think of **lifecycle callbacks** as **checkpoints** in a record's life. Whenever Rails is about to save, create, update, or delete something from the database, it gives you a chance to step in and do extra work automatically.

before_validation

This runs **just before Rails checks validations**.

You use it when you want to **fix or prepare data** so validations don't fail unnecessarily. For example, trimming spaces from a title or converting an email to lowercase before Rails checks if it's present or unique.

Example:

"Let me clean this data before Rails judges it."

before_save

This runs **right before the record is saved** (both create and update).

It's useful when you want something to happen **every time data is stored**, like setting a default value or updating a derived field.

Example:

"I'm about to store this record — do you want to adjust anything?"

after_save

This runs **after the record has been saved successfully**.

At this point, the data is already in the database, so you usually use it for **side effects** like logging, notifications, or background jobs.

Example:

"The data is saved — now you can react to it."

before_create

This runs **only when a record is created for the first time**, not on updates.

It's useful when you want to set values that should exist **only at creation**, such as generating a slug, token, or initial status.

Example:

“This record is being born — set its initial state.”

after_create

This runs **after a new record is successfully created**.

Common uses include sending a welcome email, creating associated records, or writing audit logs.

Example:

“A new record exists now — take follow-up actions.”

after_commit

This runs **after the database transaction is fully completed**.

It's the safest place to do things like sending emails or triggering external services, because Rails guarantees the data is truly saved.

Example:

“Everything is permanently saved — now it's safe to notify the outside world.”

before_destroy

This runs **just before a record is deleted**.

You use it when you want to **stop deletion** or clean things up, like preventing deletion of important records or removing dependent data manually.

Example:

“Wait — should this record really be deleted?”

```

hpghp-HP-EltteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ nano app/models/blog.rb
hpghp-HP-EltteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails console
Loading development environment (Rails 8.1.2)
blog-app(dev):001> blog = Blog.create(title: " hello rails ", content: "test", published: true)
TRANSACTION (0.2ms) BEGIN immediate TRANSACTION /*application='BlogApp'*/
Blog Create (3.7ms) INSERT INTO "blogs" ("content", "created_at", "published", "title", "updated_at") VALUES ('test', '2026-01-17 12:10:33.959187', TRUE, 'Hello rails', '2026-01-17 12:10:33.959187') RETURNING "id" /*application='BlogApp'*/
TRANSACTION (3.5ms) COMMIT TRANSACTION /*application='BlogApp'*/
=>
#<Blog:0x00074dae5d3b168
...
blog-app(dev):002>

```

In this i have used before validation helper in which i called the function which is used to trim the spaces of title and capitilize i

Use ActiveRecord methods on rails's console

```

blog-app(dev):004> Blog.where(published: true)
Blog Load (0.5ms) SELECT "blogs".* FROM "blogs" WHERE "blogs"."published" = TRUE /* loading for pp */ LIMIT 11 /*application='BlogApp'*/
=>
[#<Blog:0x00074dae592b190
id: 2,
content: "test",
created_at: "2026-01-17 12:10:33.959187000 +0000",
published: true,
title: "Hello rails",
updated_at: "2026-01-17 12:10:33.959187000 +0000">,
#<Blog:0x00074dae592b050
id: 3,
content: "demo",
created_at: "2026-01-17 12:15:02.579187000 +0000",
published: true,
title: "Welcome to ruby",
updated_at: "2026-01-17 12:15:02.579187000 +0000">]
blog-app(dev):005>

```

```

updated_at: "2026-01-17 12:15:02.579187000 +0000">]
blog-app(dev):005> Blog.find_by(title: "Hello rails")
blog-app(dev):006>
Blog Load (0.4ms) SELECT "blogs".* FROM "blogs" WHERE "blogs"."title" = 'Hello rails' LIMIT 1 /*application='BlogApp'*/
=>
#<Blog:0x00074dae589b7c0
id: 2,
content: "test",
created_at: "2026-01-17 12:10:33.959187000 +0000",
published: true,
title: "Hello rails",
updated_at: "2026-01-17 12:10:33.959187000 +0000">
blog-app(dev):007>

```

```

updated_at: "2026-01-17 10:10:45.761303000 +0000">],
blog-app(dev):007> Blog.limit(3)
blog-app(dev):008>
Blog Load (0.5ms) SELECT "blogs".* FROM "blogs" /* loading for pp */ LIMIT 5 /*application='BlogApp'*/
=>
[#<Blog:0x000074dae5897940
 id: 1,
 content: "Text",
 created_at: "2026-01-17 10:10:45.761303000 +0000",
 published: false,
 title: "Draft",
 updated_at: "2026-01-17 10:10:45.761303000 +0000">,
 #<Blog:0x000074dae5897800
 id: 2,
 content: "test",
 created_at: "2026-01-17 12:10:33.959187000 +0000",
 published: true,
 title: "Hello rails",
 updated_at: "2026-01-17 12:10:33.959187000 +0000">,
 #<Blog:0x000074dae58976c0
 id: 3,
 content: "demo",
 created_at: "2026-01-17 12:15:02.579187000 +0000",
 published: true,
 title: "Welcome to ruby",
 updated_at: "2026-01-17 12:15:02.579187000 +0000">]
blog-app(dev):009>

```

Explore order, joins, includes, eager_load, Identify the difference between includes and eager_load

Blog.order(title: :asc)

```

blog-app(dev):012> Blog.order(title: :asc)
blog-app(dev):013>
Blog Load (0.6ms) SELECT "blogs".* FROM "blogs" /* loading for pp */ ORDER BY "blogs"."title" ASC LIMIT 11 /*application='BlogApp'*/
=>
[#<Blog:0x000074dae569e120
 id: 1,
 content: "Text",
 created_at: "2026-01-17 10:10:45.761303000 +0000",
 published: false,
 title: "Draft",
 updated_at: "2026-01-17 10:10:45.761303000 +0000">,
 #<Blog:0x000074dae569dfe0
 id: 2,
 content: "test",
 created_at: "2026-01-17 12:10:33.959187000 +0000",
 published: true,
 title: "Hello rails",
 updated_at: "2026-01-17 12:10:33.959187000 +0000">,
 #<Blog:0x000074dae569dea0
 id: 3,
 content: "demo",
 created_at: "2026-01-17 12:15:02.579187000 +0000",
 published: true,
 title: "Welcome to ruby",
 updated_at: "2026-01-17 12:15:02.579187000 +0000">]
blog-app(dev):014>

```

Explore order, joins, includes, eager_load, Identify the difference between includes and eager_load

Order join :- inner join

```
blog-app(dev):032> Blog.joins(:comments)
Blog Load (0.7ms)  SELECT "blogs".* FROM "blogs" INNER JOIN "comments" ON "comments"."blog_id" = "blogs"."id" /* loading for pp */ LIMIT 11 /*application='BlogApp'*/
=>
[#<Blog:0x000074dae589a3c0
 id: 5,
 content: "This is my first blog post",
 created_at: "2026-01-17 12:21:52.961413000 +0000",
 published: true,
 title: "My first blog",
 updated_at: "2026-01-17 12:21:52.961413000 +0000">,
 #<Blog:0x000074dae589a280
 id: 5,
 content: "This is my first blog post",
 created_at: "2026-01-17 12:21:52.961413000 +0000",
 published: true,
 title: "My first blog",
 updated_at: "2026-01-17 12:21:52.961413000 +0000">]
blog-app(dev):033> █
```

Eager load :- Forces LEFT OUTER JOIN.

```
blog-app(dev):033> Blog.eager_load(:comments)
SQL (0.4ms)  SELECT DISTINCT "blogs"."id" FROM "blogs" LEFT OUTER JOIN "comments" ON "comments"."blog_id" = "blogs"."id" /* loading for pp */ LIMIT 11 /*application='BlogApp'*/
Blog Eager Load (0.8ms)  SELECT "blogs"."id" AS t0_r0, "blogs"."content" AS t0_r1, "blogs"."created_at" AS t0_r2, "blogs"."published" AS t0_r3, "blogs"."title" AS t0_r4, "blogs"."updated_at" AS t0_r5, "comments"."id" AS t1_r0, "comments"."blog_id" AS t1_r1, "comments"."body" AS t1_r2, "comments"."created_at" AS t1_r3, "comments"."updated_at" AS t1_r4 FROM "blogs" LEFT OUTER JOIN "comments" ON "comments"."blog_id" = "blogs"."id" WHERE "blogs"."id" IN (1, 2, 3, 4, 5) /* loading for pp */ /*application='BlogApp'*/
=>
[#<Blog:0x000074dae5891540
 id: 1,
 content: "Text",
 created_at: "2026-01-17 10:10:45.761303000 +0000",
 published: false,
 title: "Draft",
 updated_at: "2026-01-17 10:10:45.761303000 +0000">,
 #<Blog:0x000074dae5891400
 id: 2,
 content: "test",
 created_at: "2026-01-17 12:10:33.959187000 +0000",
 published: true,
 title: "Hello rolls",
 updated_at: "2026-01-17 12:10:33.959187000 +0000">,
 #<Blog:0x000074dae58912c0
 id: 3,
 content: "demo",
 created_at: "2026-01-17 12:15:02.579187000 +0000",
 published: true,
 title: "Welcome to ruby",
 updated_at: "2026-01-17 12:15:02.579187000 +0000">,
 #<Blog:0x000074dae5891180
 id: 4,
 content: "This is my first blog post",
 created_at: "2026-01-17 12:21:43.332083000 +0000",
 published: true,
 title: "My first blog",
 updated_at: "2026-01-17 12:21:43.332083000 +0000">,
 #<Blog:0x000074dae5891040
 id: 5,
 content: "This is my first blog post",
 created_at: "2026-01-17 12:21:52.961413000 +0000",
 published: true,
 title: "My first blog",
 updated_at: "2026-01-17 12:21:52.961413000 +0000">]
blog-app(dev):034> █
```

includes

```
blog-app(dev):036> Blog.includes(:comments).where(comments: { body: "Nice post!" })
SQL (0.5ms)  SELECT DISTINCT "blogs"."id" FROM "blogs" LEFT OUTER JOIN "comments" ON "comments"."blog_id" = "blogs"."id" WHERE "comments"."body" = 'Nice post!' /* loading for pp */ LIMIT 11 /*application='BlogApp'*/
Blog Eager Load (0.5ms)  SELECT "blogs"."id" AS t0_r0, "blogs"."content" AS t0_r1, "blogs"."created_at" AS t0_r2, "blogs"."published" AS t0_r3, "blogs"."title" AS t0_r4, "blogs"."updated_at" AS t0_r5, "comments"."id" AS t1_r0, "comments"."blog_id" AS t1_r1, "comments"."body" AS t1_r2, "comments"."created_at" AS t1_r3, "comments"."updated_at" AS t1_r4 FROM "blogs" LEFT OUTER JOIN "comments" ON "comments"."blog_id" = "blogs"."id" WHERE "comments"."body" = 'Nice post!' AND "blogs"."id" = 5 /* loading for pp */ /*application='BlogApp'*/
=>
[#<Blog:0x000074dae5718ad8
 id: 5,
 content: "This is my first blog post",
 created_at: "2026-01-17 12:21:52.961413000 +0000",
 published: true,
 title: "My first blog",
 updated_at: "2026-01-17 12:21:52.961413000 +0000">]
blog-app(dev):037> █
```

Identify the difference between `includes` and `eager_load`

The main idea

Both `includes` and `eager_load` are used to avoid the N+1 query problem when loading associated records.

The difference is how Rails loads the data under the hood.

`includes`

`includes` is the smart and flexible option.

When you use it, Rails usually runs two separate queries:

- one for the main records
- one for the associated records

Rails then matches them in memory.

Rails will only use a JOIN if it really needs to, for example when you filter or order using the associated table.

In simple words:

“Load related data efficiently, and decide the best way automatically.”

Because of this, `includes` is safer and performs better in most cases.

`eager_load`

`eager_load` always uses a JOIN, no matter what.

Rails loads everything in one big SQL query using a `LEFT OUTER JOIN`. This is useful when you must filter or sort based on the associated table.

However, the query can become heavy and slower for large datasets.

In simple words:

“Force everything to be loaded in one joined query.”

Explore seeds, and try to create 20 blogs(10 published, 10 unpublished) with comments.

```
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$ rails db:seed
Cleaning database...
Creating published blogs...
Creating unpublished blogs...
hp@hp-HP-EliteBook-840-G3:~/Desktop/Ruby_on_Rails/blog_app$
```

4. Controllers

- Add RESTful routes and actions for comment. (20 mins)
- Verify routes using rails routes
- Explore resources vs resource and namespace vs scope
- Implement a callback to initialize the resource before show action for all controllers. (10 mins)
- Implement strong parameters. (10 mins)
- Add logic to only show published blogs.
- Add an API to publish a blog.

Resources vs Resource

In Rails, resources and resource are used to define RESTful routes, but they are meant for **different kinds of data**.

resources is used when a model has **many records**. For example, blogs, comments, or users. Each record has its own ID, so the URLs include an identifier. When we use resources :blogs, Rails understands that there can be multiple blogs and automatically creates routes like /blogs, /blogs/1, /blogs/1/edit, and so on. This is the most common case in real applications.

resource (singular) is used when there is **only one instance** of something for a given context. A good example is a user profile, account settings, or a dashboard. In these cases, it doesn't make sense to have an ID in the URL because there is only one. When we use resource :profile, Rails creates routes like /profile and /profile/edit, without an ID. This makes the URL cleaner and also clearly expresses that only one record exists.

In short, use resources when you are dealing with a collection, and resource when you are dealing with a single, unique entity.

Namespace vs Scope

Both namespace and scope are used to organize routes and change URLs, but they serve slightly different purposes.

namespace is used when you want to **group routes and also separate controller logic**. It changes the URL and also tells Rails to look for controllers inside a specific folder. For example, using namespace :admin creates URLs like /admin/blogs and expects controllers to live under app/controllers/admin. This is commonly used for admin panels, APIs, or versioned routes, where the logic is clearly different from the public side of the app.

scope is used when you only want to **change the URL structure** without moving controllers into a different folder. It adds a prefix to the URL but continues using the same controllers. This is useful when the behavior is the same, but you want cleaner or more descriptive URLs.

Simply put, namespace affects both the URL and the controller location, while scope only affects the URL.

Callback to Initialize a Resource Before show Action

In Rails, a callback is a method that runs automatically before or after a controller action. Callbacks are commonly used to avoid repeating the same code in multiple actions. One frequent example is loading a record from the database before showing it on the screen.

before_action :set_blog, only: [:show, :edit, :update, :destroy]

When an application has show, edit, update, or destroy actions, each of them usually needs to find the same record using an ID from the URL. Writing this lookup logic again and again makes the controller harder to read and maintain. A callback helps by moving this repeated logic into one place and running it automatically when needed.

In short, callbacks improve readability, reduce duplication, and make controllers easier to maintain.

How This Works Internally

1. A request comes in (for example: GET /blogs/3)
 2. Rails sees the `before_action`
 3. `set_blog` runs first
 4. `@blog` is loaded from the database
 5. The show action runs
 6. The view uses `@blog` to render the page
-

Strong Parameters

Strong parameters are a Rails mechanism used to control **which data is allowed to be saved to the database**. When a form is submitted, Rails receives all the data as a parameters hash. By default, this data cannot be directly used to create or update records because it may contain extra or unsafe fields.

Without strong parameters, a user could manually send additional fields that were never intended to be updated, such as admin flags or foreign keys. Strong parameters solve this problem by forcing developers to explicitly declare which attributes are permitted.

In simple terms, strong parameters act like a filter. Only the approved fields pass through, and everything else is ignored. This keeps the application secure and prevents accidental or malicious data changes.

Example:

```
private

def blog_params

  params.require(:blog).permit(:title, :body, :published)

end
```


`require(:blog)` ensures the data is wrapped under `blog`

`permit` lists the fields allowed to be saved

Logic to Show Only Published Blogs

In most real applications, not all content should be visible to users. Some blogs may be drafts or unfinished and should stay hidden until they are ready. To handle this, the application needs a simple rule: **only blogs marked as published should be shown to users.**

Steps to Show Only Published Blogs

1. Ensure the `blogs` table has a `published` boolean column.
2. Add a `published` scope in the `Blog` model to filter published records.
3. Use the `published` scope in the `blogs index` action to list only published blogs.
4. Update the `show` action to fetch blogs only from the `published` scope.
5. Verify that unpublished blogs are not accessible via direct URLs.

